3522 BULUT BİLİŞİM DERSİ

PROJE 2

RAPORU

Gerçek Zamanlı Veri Akışı ve İşleme

WebSocket + Google Cloud Pub/Sub Uygulaması

Kullanılan Teknolojiler

- Node.js (Backend Runtime)
- WebSocket / Binance API
- Google Cloud Pub/Sub
- IAM Service Account

Proje Özeti

Binance borsasından WebSocket üzerinden anlık BTC/USDT verisi alınmış; Google Cloud Pub/Sub aracılığıyla buluta aktarılmış ve asenkron tüketilmiştir.

GitHub Proje Reposu
github.com/SALIM-20291313/Proje-2

Hazırlayan	Ders Sorumlusu	Öğretim Yılı / Tarih
SALİM CAN ŞEN	**ENVER BAĞCI**	**2025 – 2026**
20291313		Nisan 2026

İÇİNDEKİLER

1.	**Giriş ve Projenin Amacı**	**3**
2.	**Kullanılan Teknolojiler**	**4**
3.	**Proje Mimarisi**	**4**
3.1	Uçtan Uca Veri Akış Şeması	4
3.2	Katman Açıklamaları	5
3.3	Mimarinin Tasarım Kararları	5
4.	**Google Cloud Projesi Kurulumu**	**6**
4.1	Proje Oluşturma ve API Aktivasyonu	6
4.2	Topic ve Subscription Yapılandırması	6
4.3	IAM ve Servis Hesabı Güvenliği	6
5.	**Uygulama Kodu ve Çalışma Mantığı**	**7**
5.1	publisher.js — Veri Yayıncısı	7
5.2	subscriber.js — Veri Tüketicisi	8
6.	**Pub/Sub Kullanımının Avantajları**	**9**
7.	**Projenin Çalıştırılması**	**9**
8.	**Sonuç ve Değerlendirme**	**10**

1. Giriş ve Projenin Amacı

Bulut bilişim alanında gerçek zamanlı veri işleme, modern yazılım mimarilerinin temel taşlarından birini oluşturmaktadır. Bu proje kapsamında, kripto para piyasasından elde edilen anlık finansal veri akışını yönetmek amacıyla bulut tabanlı bir mesajlaşma ve yayın altyapısı tasarlanmıştır.

Projenin temel çıkış noktası şu soruya yanıt aramaktır: Dışarıdan sürekli akan bir veri akışı, aracı bir bulut katmanı üzerinden nasıl güvenilir, ölçeklenebilir ve kayıpsız biçimde birden fazla tüketiciye ulaştırılabilir?

Projenin Temel Hedefleri

Gerçek zamanlı veri akışı yönetimi • WebSocket protokolü ile dış API entegrasyonu • Google Cloud Pub/Sub mesaj kuyruğu kullanımı • IAM ile servis hesabı güvenliği • Ölçeklenebilir yayın-abonelik mimarisi kurulumu • Asenkron ve olay tabanlı mimari deneyimi

Bu hedefler doğrultusunda Node.js tabanlı iki ayrı uygulama geliştirilmiştir. publisher.js Binance'ın halka açık WebSocket API'sine bağlanarak BTC/USDT paritesine ait anlık işlem verilerini alır ve Google Cloud Pub/Sub servisine iletir. subscriber.js ise bu servisten gelen mesajları dinleyerek işler.

2. Kullanılan Teknolojiler

Projenin geliştirilmesinde aşağıdaki teknoloji ve bileşenler kullanılmıştır. Her bileşen belirli bir katmanı üstlenmekte olup birlikte uçtan uca bir veri işleme pipeline'ı oluşturmaktadır.

Teknoloji / Bileşen	Kategori	Kullanım Amacı
Node.js v18+	Backend Runtime	WebSocket bağlantısı ve Pub/Sub SDK çalışma ortamı
ws (npm paketi)	WebSocket İstemcisi	Binance API'sine gerçek zamanlı TCP bağlantısı
@google-cloud/pubsub	GCP Resmi SDK	Topic'e mesaj yayınlama ve abonelik yönetimi
Google Cloud Pub/Sub	Bulut Mesaj Kuyruğu	Asenkron, ölçeklenebilir, güvenilir veri iletim altyapısı
GCP IAM	Kimlik & Erişim Yönetimi	Least Privilege prensibi ile güvenli GCP erişimi
key.json	Kimlik Anahtarı	JSON tabanlı service account yetkilendirmesi
Binance WS API	Gerçek Zamanlı Veri	BTC/USDT anlık işlem akışı (trade stream)

Node.js, asenkron I/O yapısı ve olay güdümlü modeli sayesinde WebSocket bağlantısı gibi sürekli akan veri akışları için ideal bir çalışma ortamı sağlar. @google-cloud/pubsub paketi ise Google'ın resmi SDK'sı olarak GCP kimlik doğrulaması ve Pub/Sub operasyonlarını tek bir kütüphane altında yönetir.

3. Proje Mimarisi

Sistemin uçtan uca mimarisi üç ana katmandan oluşmaktadır: Veri Kaynağı Katmanı, Mesaj Aracısı Katmanı ve Tüketici Katmanı. Her katman birbirinden bağımsız çalışabilir; bu da sistemin ölçeklenmesini ve bakımını kolaylaştırır.

3.1 Uçtan Uca Veri Akış Şeması

Binance WebSocket API	wss://stream.binance.com:9443/ws/btcusdt@trade
↓ Gerçek Zamanlı Veri Akışı	JSON formatında BTC/USDT işlem verisi (fiyat, miktar, zaman)
publisher.js (Node.js)	WebSocket'ten veri alır → JSON parse → Buffer'a çevirir → Publish eder
↓ Pub/Sub Publish()	Buffer veri, service account kimliğiyle topic'e iletilir
Google Cloud Pub/Sub	binance-veri-akisi | Güvenilir mesaj kuyruğu & kalıcı depolama
↓ Subscription Pull	binance-veri-akisi-sub aboneliği üzerinden olay tabanlı tüketim
subscriber.js (Node.js)	Mesajı decode eder → loglar → message.ack() ile onaylar

3.2 Katman Açıklamaları

Katman 1 — Veri Kaynağı

Binance, dünyanın en büyük kripto para borsalarından biridir. Binance'ın WebSocket Streams API'si kayıt gerektirmeksizin halka açık olarak sunulmaktadır. btcusdt@trade kanalı gerçek zamanlı her işlemi milisaniye hassasiyetinde yayınlar. Her mesaj; işlem fiyatı (p), miktarı (q), işlem zamanı (T) ve alıcı/satıcı taraf bilgisini içerir.

Binance API Endpoint

wss://stream.binance.com:9443/ws/btcusdt@trade Örnek mesaj alanları: { e: 'trade', p: '67250.00', q: '0.0021', T: 1714120000000 }

Katman 2 — Mesaj Aracısı (Google Cloud Pub/Sub)

Google Cloud Pub/Sub, Google'ın yönetilen mesaj kuyruğu ve yayın servisidir. Publisher-Subscriber modelini uygular: bir topic'e mesaj gönderen herhangi bir sayıda publisher olabilirken aynı topic'i dinleyen herhangi bir sayıda subscriber olabilir. Mesajlar, tüm aboneler tarafından alınana dek veya retention süresi dolana dek saklanır.

Bu projede tek bir publisher (publisher.js) ve tek bir subscriber (subscriber.js) mevcuttur. Ancak mimari genişletildiğinde aynı topic'e birden fazla uygulama aynı anda abone olabilir: veritabanı yazıcısı, analitik motoru, mobil bildirim servisi gibi.

Katman 3 — Tüketici (subscriber.js)

Tüketici katmanı, Pub/Sub'dan gelen mesajları olay tabanlı olarak işler. message.ack() çağrısı GCP'ye mesajın başarıyla alındığını bildirir ve mesajın kuyruktan çıkarılmasını sağlar. Eğer ack() çağrılmazsa GCP mesajı tekrar teslim etmeye çalışır — bu at-least-once delivery garantisini oluşturur.

3.3 Mimarinin Tasarım Kararları

Projenin mimarisinde bilinçli olarak alınan bazı tasarım kararları sistemin genel kalitesini doğrudan etkiler:

- **Asenkron İletim:** Pub/Sub, publisher ve subscriber'ı zamansal olarak ayırır. Publisher veri gönderirken subscriber'ın çalışıyor olması şart değildir.
- **Least Privilege (En Az Yetki):** IAM service account'a yalnızca Pub/Sub Admin rolü verilmiştir. GCP kaynaklarının geri kalanına erişim yoktur.
- **Buffer Dönüşümü:** JSON veri, publish öncesi Buffer.from() ile binary'ye çevrilir. Bu GCP SDK'nın zorunlu kıldığı formattır.
- **Try-Catch Sarmalama:** Pub/Sub publish işlemi try-catch ile sarmalanmıştır. Bir yayın hatası uygulamanın çökmesine neden olmaz.
- **Event-Driven Mimari:** Her iki uygulama da event listener kullanır. Bu yapı CPU'yu boş beklemeyle tüketmez.

4. Google Cloud Projesi Kurulumu

Projenin bulut altyapısı Google Cloud Platform üzerinde yapılandırılmıştır. Bu bölümde adım adım kurulum süreci açıklanmaktadır.

4.1 Proje Oluşturma ve API Aktivasyonu

GCP Proje Kimliği (Project ID)
project-6160b355-f911-4b6a-95c

GCP Console üzerinden yeni bir proje oluşturulduktan sonra APIs & Services → Enable APIs & Services bölümünden Cloud Pub/Sub API aranarak aktive edilmiştir. Bu adım olmadan hiçbir Pub/Sub işlemi yapılamaz.

4.2 Topic ve Subscription Yapılandırması

Bileşen	Değer / Açıklama
Topic ID	`binance-veri-akisi`
Subscription ID	`binance-veri-akisi-sub`
İletim Tipi	`Asenkron, olay tabanlı (event-driven)`
Mesaj Saklama	`7 gün (varsayılan GCP süresi)`
Onay Mekanizması	`message.ack() — at-least-once delivery garantisi`

4.3 IAM ve Servis Hesabı Güvenliği

Uygulamanın Google Cloud kaynaklarına programatik olarak erişebilmesi için bir Service Account oluşturulmuştur.

- IAM ekranından yeni bir service account tanımlanmıştır
- **Atanan rol:** Pub/Sub Admin — yalnızca Pub/Sub kaynaklarına tam erişim
- Kimlik doğrulama için JSON anahtar dosyası oluşturulmuş ve key.json adıyla proje dizinine yerleştirilmiştir
- key.json içinde proje kimliği, private key ve client email bilgileri bulunur

Güvenlik Uyarısı
key.json dosyası asla Git reposuna eklenmemelidir. .gitignore ile sürüm kontrolü dışında tutulmalıdır.

5. Uygulama Kodu ve Çalışma Mantığı

Proje iki ayrı Node.js betiğinden oluşmaktadır. Bu iki betiğin sorumluluklarının birbirinden net biçimde ayrılması (Separation of Concerns), sistemin bakımını ve genişletilmesini kolaylaştırmaktadır.

5.1 publisher.js — Veri Yayıncısı

publisher.js sistemin veri giriş noktasıdır. Binance WebSocket akışına bağlanır, gelen her trade mesajını parse eder ve Google Cloud Pub/Sub topic'ine iletir.

1	**GCP Kimlik Doğrulaması** PubSub nesnesi oluşturulurken projectId ve keyFilename parametreleri verilir. SDK, key.json dosyasını okuyarak OAuth2 üzerinden otomatik kimlik doğrulaması gerçekleştirir.
2	**WebSocket Bağlantısı (Binance API)** ws kütüphanesi ile wss://stream.binance.com:9443/ws/btcusdt@trade endpointine kalıcı bir TCP bağlantısı açılır. Bağlantı kesilmeden BTC/USDT işlem akışı dinlenir.
3	**Mesaj Alma ve Parse İşlemi** WebSocket'ten gelen mesaj ham string formatındadır. toString() ile string'e, JSON.parse() ile JavaScript nesnesine dönüştürülür. Fiyat ve hacim bilgisi terminale loglanır.
4	**Buffer'a Çevirme ve Publish** JSON nesne Buffer.from(JSON.stringify(trade)) ile byte dizisine çevrilir. GCP SDK'nın publish() metodu bu buffer'ı topic'e iletir. İşlem async/await ile yürütülür.
5	**Hata Yönetimi ve Dayanıklılık** Publish işlemi try-catch bloğuna sarılmıştır. close ve error olayları ayrıca dinlenir. Herhangi bir hata uygulamanın çökmesine neden olmaz.

publisher.js — Tam Kod

```
// publisher.js — Binance WS → Google Cloud Pub/Sub
const { PubSub }   = require('@google-cloud/pubsub');
const WebSocket    = require('ws');
```

```
const PROJECT_ID   = 'project-6160b355-f911-4b6a-95c';
const TOPIC_NAME   = 'binance-veri-akisi';
const WS_URL       = 'wss://stream.binance.com:9443/ws/btcusdt@trade';

const pubSubClient = new PubSub({ projectId: PROJECT_ID, keyFilename: './key.json'
});
const topic        = pubSubClient.topic(TOPIC_NAME);
const ws           = new WebSocket(WS_URL);

ws.on('open', () => console.log('[WS] Binance bağlantısı kuruldu.'));

ws.on('message', async (data) => {
  const trade     = JSON.parse(data.toString());
  console.log(`[TRADE] Fiyat: ${trade.p}  Miktar: ${trade.q}`);
  const msgBuffer = Buffer.from(JSON.stringify(trade));
  try {
    const msgId = await topic.publish(msgBuffer);
    console.log(`[PUB] Mesaj yayınlandı. ID: ${msgId}`);
  } catch (err) { console.error('[PUB] Hata:', err.message); }
});

ws.on('close', () => console.log('[WS] Bağlantı kapandı.'));
ws.on('error', (e) => console.error('[WS] Hata:', e.message));
```

5.2 subscriber.js — Veri Tüketicisi

subscriber.js Pub/Sub kuyruğundaki mesajları olay tabanlı olarak dinleyen ve işleyen Node.js betiğidir. Dış veri kaynağıyla doğrudan ilişkisi yoktur; yalnızca GCP üzerindeki abonelik kanalını dinler.

Subscriber'ın publisher'dan tamamen bağımsız çalışması sistemin en kritik mimari özelliğidir. Publisher çalışmasa da subscriber mesajları kuyruktan okumaya devam edebilir; subscriber çöküp yeniden başlasa da okunmamış mesajlar kaybolmaz.

- subscription.on('message') ile her gelen mesajda callback tetiklenir
- message.data (Buffer) → toString() → JSON.parse() dönüşüm zinciri uygulanır
- İşlenen her mesaj 'Buluttan Gelen Veri:' başlığıyla terminale loglanır
- message.ack() çağrılarak GCP'ye mesajın başarıyla işlendiği bildirilir
- Eğer ack() çağrılmazsa GCP mesajı yeniden teslim etmeye çalışır (at-least-once garantisi)

subscriber.js — Tam Kod

```
// subscriber.js — Google Cloud Pub/Sub → Konsol
const { PubSub }  = require('@google-cloud/pubsub');

const PROJECT_ID  = 'project-6160b355-f911-4b6a-95c';
const SUB_NAME    = 'binance-veri-akisi-sub';

const pubSubClient = new PubSub({ projectId: PROJECT_ID, keyFilename: './key.json'
});
const subscription = pubSubClient.subscription(SUB_NAME);
```

```
console.log(`[SUB] Abonelik dinleniyor: ${SUB_NAME}`);

subscription.on('message', (message) => {
  const trade = JSON.parse(message.data.toString());
  console.log('Buluttan Gelen Veri:', {
    fiyat  : trade.p,
    miktar : trade.q,
    zaman  : new Date(trade.T).toISOString()
  });
  message.ack();   // Mesajı onayla, kuyruktan çıkar
});

subscription.on('error', (err) => console.error('[SUB] Hata:', err.message));
```

6. Pub/Sub Kullanımının Avantajları

Bu projede verilerin doğrudan WebSocket üzerinden tek bir tüketiciye iletilmesi yerine Google Cloud Pub/Sub aracılığıyla buluta aktarılması sisteme çok boyutlu avantajlar kazandırmaktadır.

Kriter	Doğrudan WebSocket	Google Cloud Pub/Sub
Ölçeklenebilirlik	Tek istemci, sınırlı tüketim	Yüzlerce mikroservis aynı anda tüketebilir
Güvenilirlik	Bağlantı kopunca veri yok olur	Mesajlar kuyrukta bekler, kayıpsız teslim
Hata Toleransı	Abone çökerse veri geri alınamaz	Abone yeniden başlayınca kaldığı yerden devam
İzlenebilirlik	Çok sınırlı, manuel log gerektirir	GCP konsolundan anlık monitoring & alerting
Mesaj Garantisi	Yok — fire and forget	At-least-once delivery, message.ack() onayı

Bu avantajların somut örneği: Sistem üretimde çalışırken subscriber bir sunucu güncellemesi nedeniyle 10 dakika kapalı kalır. Doğrudan WebSocket mimarisinde bu süredeki tüm BTC/USDT verileri kaybolur. Pub/Sub mimarisinde ise mesajlar kuyrukta bekler; subscriber yeniden başladığında tüm verileri sırayla işler.

7. Projenin Çalıştırılması

Sistemi yerel ortamınızda ayağa kaldırmak için aşağıdaki adımları sırayla uygulayınız. publisher.js ve subscriber.js birbirinden bağımsız terminal oturumlarında çalıştırılmalıdır.

Ön Koşul

publisher.js ve subscriber.js ile aynı dizinde key.json kimlik dosyasının bulunduğundan emin olunuz. Ayrıca Node.js v18+ yüklü olmalıdır.

Adım 1 — Bağımlılıkların Kurulumu

```
npm install
```

Adım 2 — Publisher'ı Başlatma (Terminal 1)

```
node publisher.js
```

Adım 3 — Subscriber'ı Başlatma (Terminal 2)

```
node subscriber.js
```

publisher.js çalıştıktan sonra terminalde gerçek zamanlı BTC/USDT trade mesajları loglanmaya başlar. subscriber.js ise Pub/Sub kuyruğundan okuduğu her mesajı 'Buluttan Gelen Veri:' başlığıyla ekrana yazar.

8. Sonuç ve Değerlendirme

Bu proje kapsamında gerçek zamanlı kripto para verisi akışını bulut üzerinde yönetmek için eksiksiz ve çalışan bir pipeline başarıyla kurulmuştur. Binance WebSocket API'si aracılığıyla anlık BTC/USDT işlem verisi elde edilmiş; Google Cloud Pub/Sub'ın yayın-abonelik modeli sayesinde bu veriler ölçeklenebilir, güvenilir ve kayıpsız biçimde buluta aktarılmış ve tüketilmiştir.

Proje; IoT ve finansal teknoloji uygulamalarında yaygın olarak kullanılan endüstriyel mimari kalıplarını uygulamalı olarak deneyimleme imkânı sunmuştur. Least Privilege ile IAM güvenliği, mesaj kuyruğuyla hata toleransı, asenkron programlamayla ölçeklenebilirlik ve event-driven mimariyle kaynak verimliliği bu proje üzerinden somut biçimde pekiştirilmiştir.

Sistem ilerleyen aşamalarda BigQuery entegrasyonu ile veri analitiği, Cloud Functions ile sunucusuz işleme veya Dataflow ile streaming ETL pipeline'larına genişletilebilir.

GitHub Proje Reposu

Proje kodları, rapor ve çalışma logları aşağıdaki Git reposunda sürüm kontrolü ile yönetilmiştir.

https://github.com/SALIM-20291313/Proje-2